



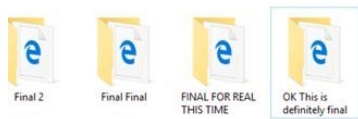
Git

для командной разработки

Потапова А.
a.potapova8@g.nsu.ru

17.02.2026

Зачем вообще нужны системы контроля версий?

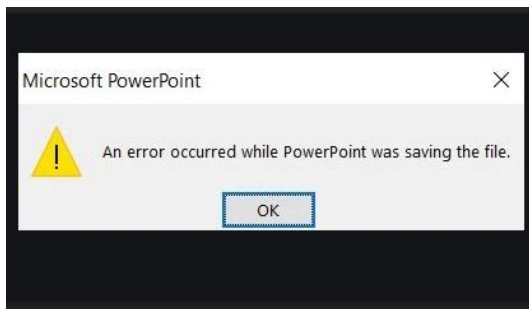


Зачем вообще нужны системы контроля версий?

1. Машина времени



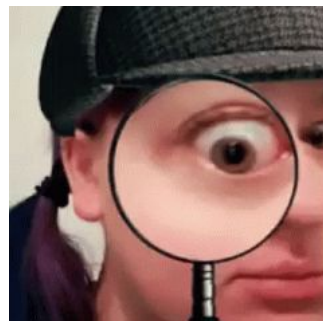
2. Защита от потери данных



3. Работа в команде



4. Ответственность и история



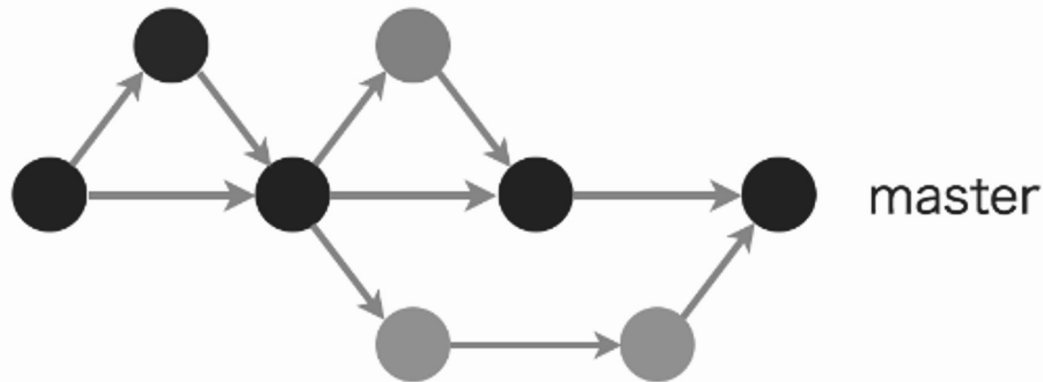
Как работает git

Проект — граф изменений

Коммит — узел графа.

- имеет хэш
- хранит изменения в файлах и метаданные (автор, дата, сообщение)

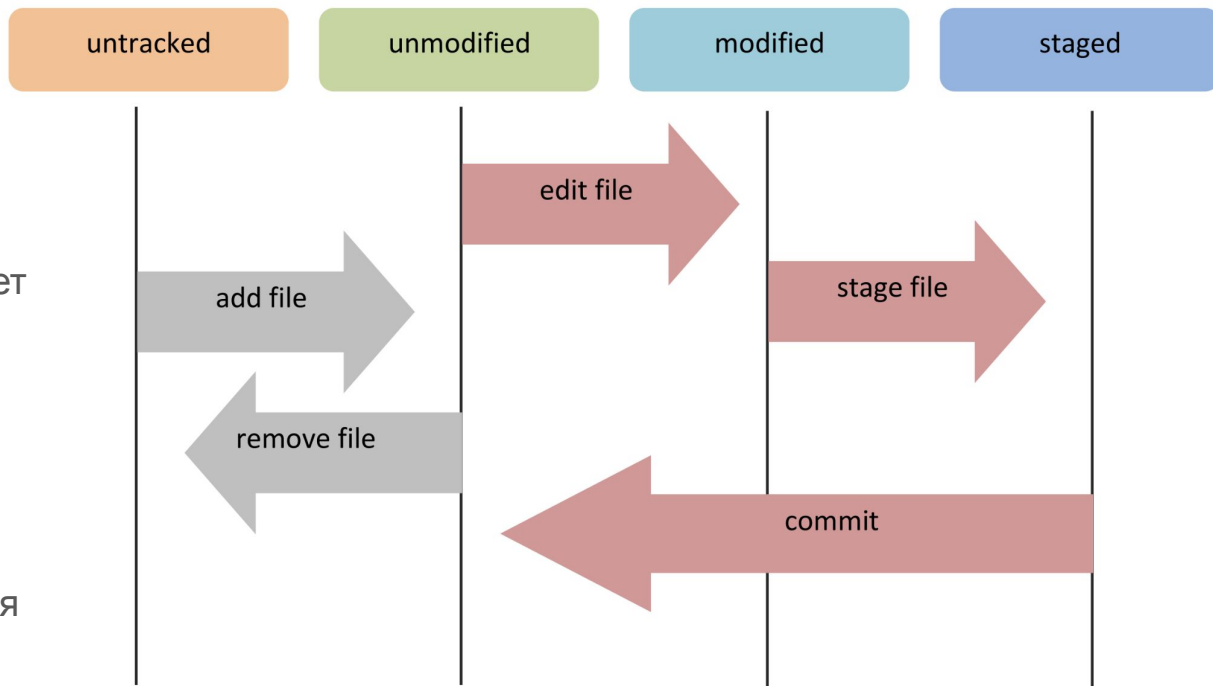
Имеет ссылку на родительский коммит (или несколько). Именно эти ссылки выстраивают историю.



Стрелки показывают историю изменений.
Зависимости будут инвертированы(!)

Состояния файлов или как сделать коммит

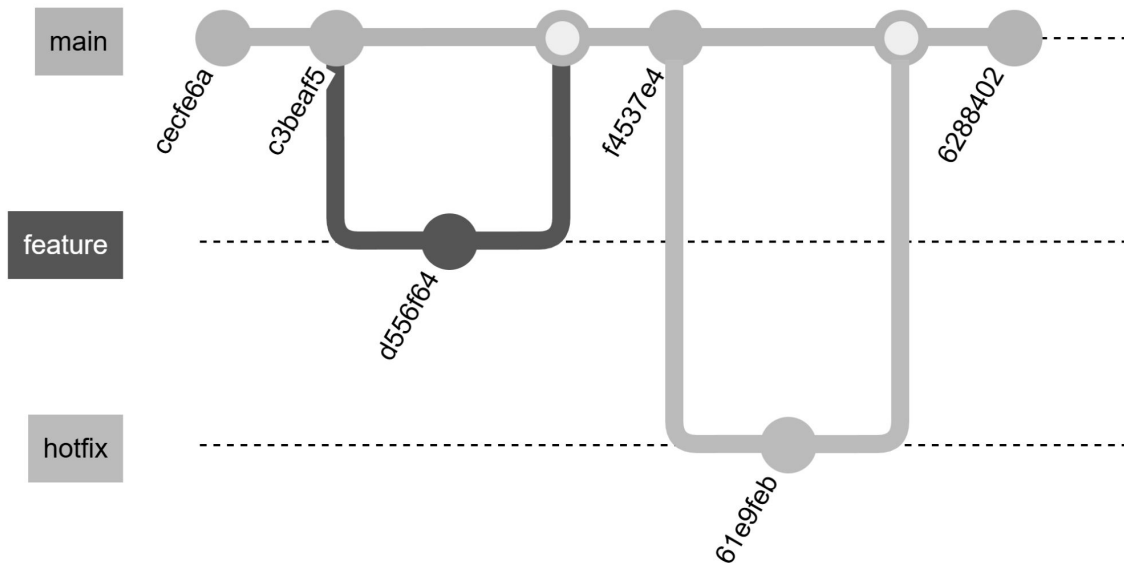
1. **Untracked**
файл новый, Git ещё не отслеживает его.
2. **Unmodified (Committed)**
файл отслеживается, но изменений нет (соответствует последнему коммиту).
3. **Modified**
файл изменён, но ещё не добавлен в индекс.
4. **Staged**
изменения подготовлены для коммита (файлы проиндексированы).



Ветвление

Ветка — это последовательность коммитов, при последовательном применении которых мы можем восстановить состояние проекта в любой момент времени.

Технически же это просто указатель на коммит.



Создадим ветку и
добавим пару коммитов

Демо

```
git branch <branch-name>
```

```
git checkout -b <branch-name>
```

```
git log
```

```
git add <files>
```

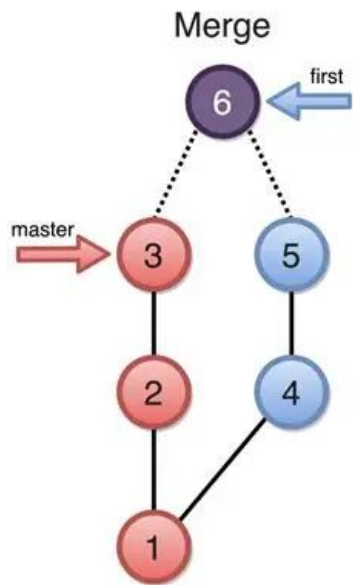
```
git reset HEAD <file>
```

```
git commit -m <commit-msg>
```

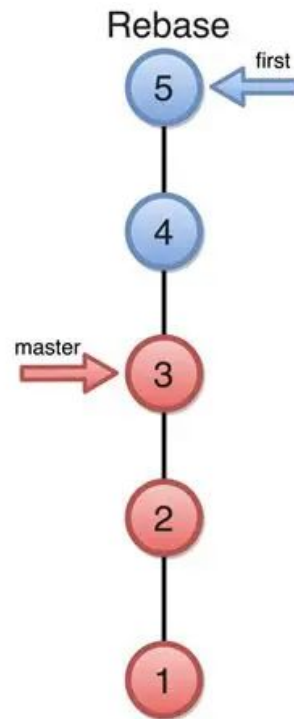
```
git diff
```

```
git status
```


Слияние



Два типа слияния:
merge и **rebase**



Когда какое слияние использовать?

Merge если:

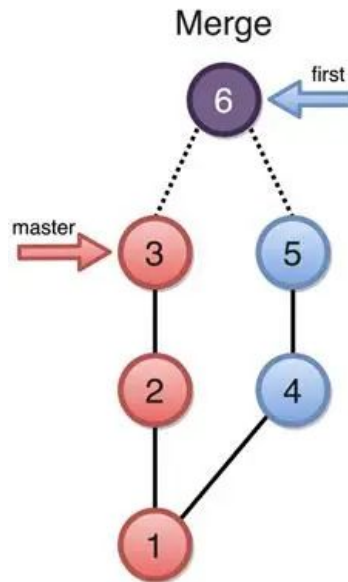
Вливаем готовую ветку в основную (main, develop) — например, через Pull Request

Плюсы:

- Однократное разрешение конфликтов
- Сохранение реальной истории
- Безопасность: merge не переписывает существующие коммиты

Минусы:

- Захламление истории
- Часто сложно читать историю изменений



Когда какое слияние использовать?

Rebase если:

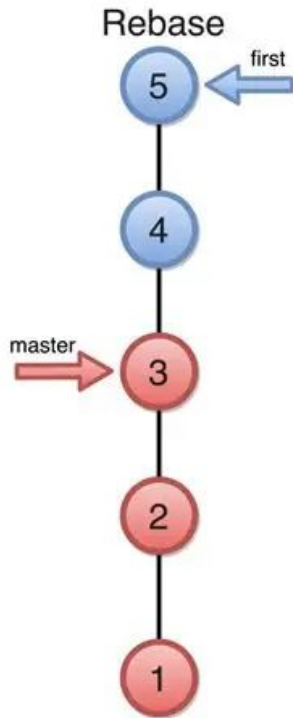
Обновляем личную ветку перед созданием Pull Request

Плюсы:

- Линейная история

Минусы:

- Переписывание истории
- Конфликт на каждый коммит



Смержим в main
изменения из нашей
ветки

Выполним rebase чтобы
обновить нашу ветку в
соответствии с main

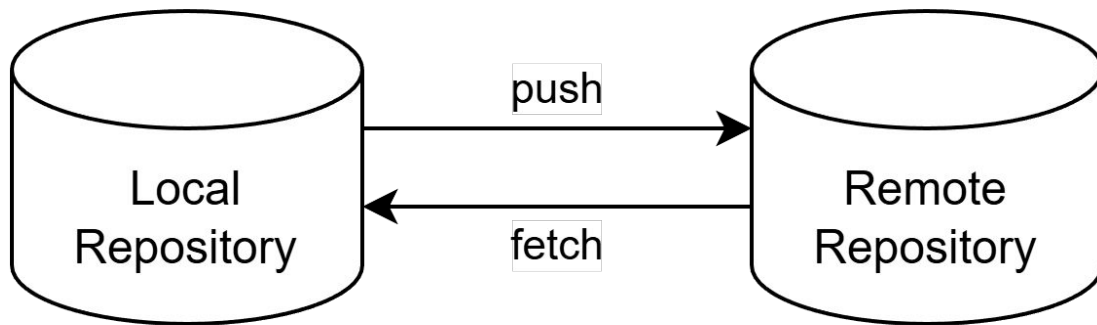
Демо

```
git merge <source-branch>
```

```
git rebase <target-branch>
```

Удаленный репозиторий

Это точно такая же копия графа, расположенная на сервере, и мы синхронизируем с ней свои локальные изменения.



Origin — стандартное имя удалённого репозитория

Синхронизация

git fetch



Забирает новые коммиты с сервера, но **не сливает их с вашими локальными ветками.**

Обновляет удалённые ветки (origin/main, origin/feature).

git pull



git fetch + *git merge*

Сначала забирает новые коммиты, затем **пытается влить удалённую ветку** в вашу текущую локальную ветку.

Конфликты разрешаются как при обычном merge.

git push



Отправляет ваши новые коммиты на сервер.

Если удалённая ветка продвинулась вперед, Git отклонит push с ошибкой: «non-fast-forward».

Поэтому **сначала делаем pull**, объединяем изменения, **а затем снова push.**

Подтянем изменения с
удаленного репозитория

Запустим свои
изменения

Демо

```
git clone <url>
```

```
git remote -v
```

```
git fetch <remote>
```

```
git pull <remote>
```

```
git push <remote> <branch>
```

Командная работа

Процесс выполнения задачи

1. Получение задачи и обновление локальной копии

Перед началом работы нужно убедиться, что ваша локальная основная ветка **синхронизирована** с удаленной.

2. Создание ветки для задачи

Каждая задача (фича, багфикс, рефакторинг) должна выполняться в **отдельной ветке(!!!)**.

Название ветки обычно отражает суть задачи и может включать ее идентификатор.

Например, `feature/123-add-login`.

Процесс выполнения задачи

3. Разработка и коммиты

В процессе работы **делайте коммиты** — маленькие, логически завершённые изменения. Каждый коммит должен быть осмысленным.

4. Периодическая синхронизация с основной веткой

Пока вы работаете, коллеги могли уже влить свои изменения в main. Чтобы ваша ветка не устарела и чтобы избежать больших конфликтов в будущем, регулярно подтягивайте обновления.

Процесс выполнения задачи

5. Подготовка к ревью

Перед тем как создавать Pull Request, убедитесь, что ветка содержит все нужные изменения и актуальна.

1. Все коммиты логичны, сообщения понятны.
2. Ветка не содержит лишних файлов
3. Проект собирается и проходит тесты

6. Создание Pull Request (Merge Request)

Через интерфейс GitHub/GitLab создаете PR из вашей ветки в основную ветку.

Что указать в PR:

1. Название
2. Описание: что сделано, как тестировать, ссылка на задачу в таскборде.



Лирическое отступление



Понятное дело, что данный план – это просто свод указаний, а не жестких законов.

Вы можете подстраивать этот процесс под себя, вводить свои правила или игнорировать что то из вышеперечисленного.

Главное, чтобы это решение было **осознанным**, а разработка в команде – **прозрачной**.

Best practices

Best practices. Коммиты

- **Каждый коммит — логическая единица.**
Не мешай исправление опечатки с новой фичей. Если нужно откатить одно изменение, не потеряется второе.
- **Пиши осмысленные сообщения.**
Плохо: `fix`, `asdf`, `upd`
Хорошо: `feat: add password reset form`, `fix: handle empty state in profile`
- **Не коммить сырые/несобранные изменения.**
Убедись, что код компилируется/проходит тесты перед коммитом.

Best practices. Ветки

- **Одна ветка — одна задача.**

Название отражает суть: `feature/123-login`, `bugfix/header-disappear`. В именах используй принятый в команде префикс (`feature`, `bugfix`, `hotfix`, `chore...`).

- **Никогда не работай прямо в main (или develop) (!!!).**

Даже для «маленькой правки» создай ветку. Это защитит от случайных поломок и упростит откат.

- **Удаляй ветки после вливания.**

Локально: `git branch -d feature/xxx`.

Удалённо: `git push origin --delete feature/xxx`.

Синхронизация и конфликты

- **Перед началом новой задачи обнови main.**
Используй `git pull origin main`. Так ты стартуешь с актуальной версии.
- **Регулярно подтягивай изменения в свою ветку.**
Используй `git pull --rebase`. Это уменьшит объём конфликтов в конце.
- **Rebase или merge для обновления?**
В личной ветке лучше rebase — история линейна и красива. В общей — лучше merge.
- **Не бойся конфликтов.**
Просто открой файл, оставь нужные строки и убери маркеры. После разрешения не забудь `git add` и продолжи операцию (`git rebase --continue` или `git merge --continue`).
- **Если запутался — всегда можно откатиться.**
`git rebase --abort`, `git merge --abort` отменяют слияние.

Лайфхаки

.gitignore

- папки IDE (.idea, .vscode),
- зависимости (node_modules),
- бинарники,
- логи,
- файлы с секретами (.env).

```
.gitignore x
1 .gitignore
2
```



Примеры готовых .gitignore для разных языков можно взять на gitignore.io.

Исправление коммитов



Git commit
message
actually summarizes
what you
changed in the code



"Small
changes"

Забыл файл? Опечатка в сообщении?

Добавь изменения в индекс и выполни `git commit --amend`. Новый коммит заменит старый.

В любых непонятных ситуациях – reflog ⚠

Даже если ты потерял коммит (например, сбросил `reset` не туда), `git reflog` покажет все действия, и можно вернуться в любое состояние.

Force-push

Если всё же надо перезаписать историю, используй `git push --force-with-lease` вместо `--force`. Это безопаснее — git проверит, что чужие изменения случайно не затрут.



У меня в ветке много мелких коммитов, хочу сделать один красивый перед PR 🧩

Используй интерактивный rebase:

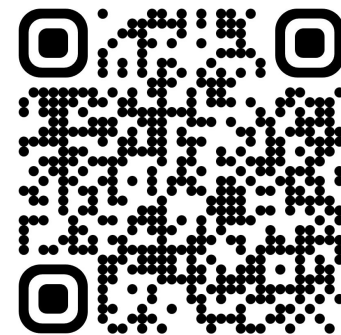
`git rebase -i HEAD~N` где N - количество коммитов.

Выбери `squash` или `fix` для объединения.

- `squash`: объединяет коммит с предыдущим, при этом объединяя сообщения коммитов.
- `fix`: *то же самое, но* сообщения коммитов не объединяются, но остается только сообщение конечного коммита



спасибо за
внимание



Потапова А.
a.potapova8@g.nsu.ru